

Participación 1

Transfer Learning para clasificación de imágenes

Integrantes:

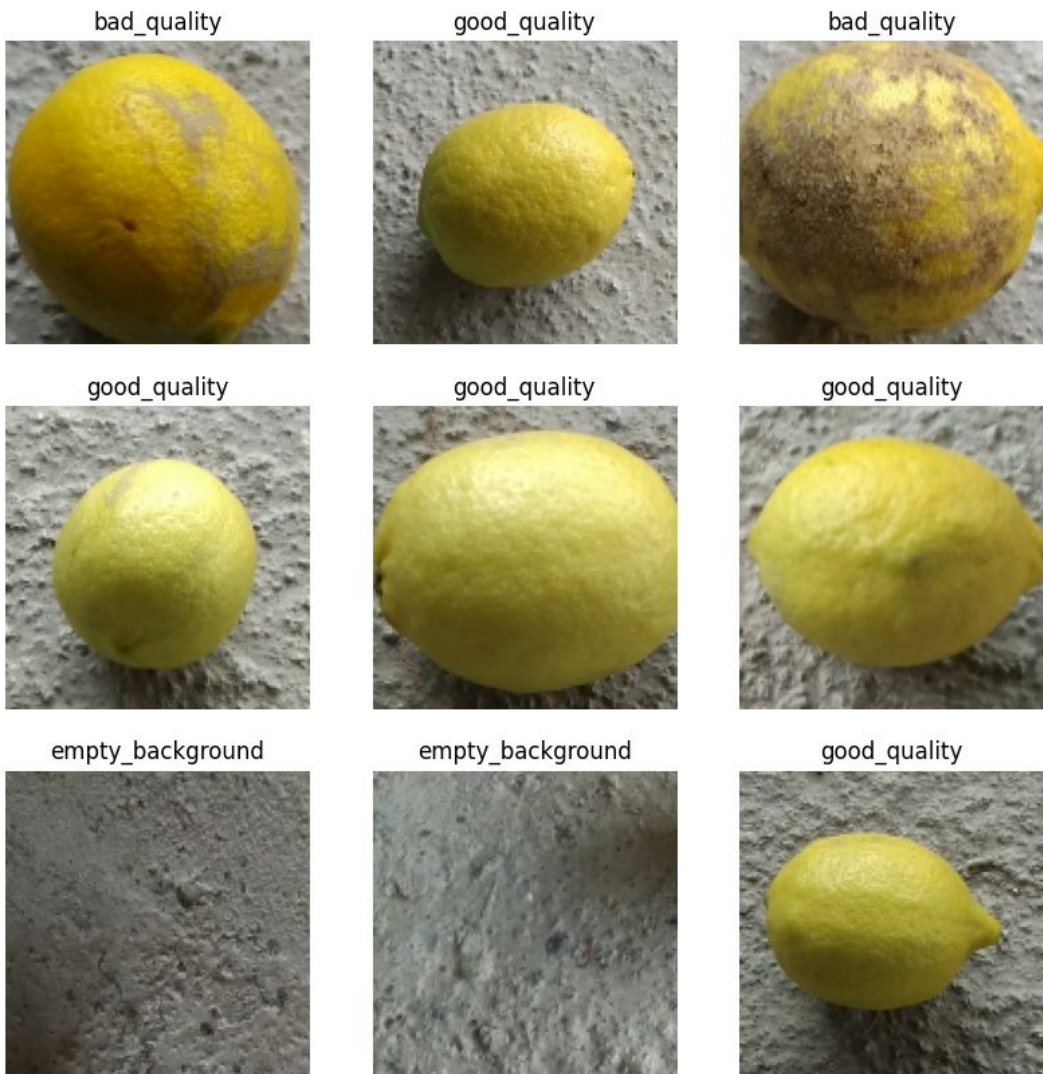
- Yobel Bañes
- Angel Eduardo Hinocho Jove
- Rommel Paredes Banda
- Nicolas Torres Valencia

CALIDAD DE LIMONES

Ejemplos de imágenes del conjunto de entrenamiento



Ejemplos de imágenes del conjunto de validación



PREGUNTA 1

```
1 from tensorflow.keras.applications import EfficientNetB0
2 from tensorflow.keras import layers, models
3
4 base_model = EfficientNetB0(
5     weights='imagenet',
6     include_top=False,
7     input_shape=(128, 128, 3)
8 )
9 base_model.trainable = False
10
11 model = models.Sequential([
12     base_model,
13     layers.GlobalAveragePooling2D(),
14     layers.Dropout(0.3),
15     layers.Dense(len(class_names), activation='softmax')
16 ])
17
18 model.compile(optimizer='adam',
19               loss='categorical_crossentropy',
20               metrics=['accuracy'])
21
22 model.summary()
```

Layer (type)	Output Shape	Param #
efficientnetb0 (Functional)	(None, 4, 4, 1280)	4,049,571
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None, 1280)	0
dropout_2 (Dropout)	(None, 1280)	0
dense_2 (Dense)	(None, 3)	3,843

Total params: 4,053,414 (15.46 MB)

Trainable params: 3,843 (15.01 KB)

Non-trainable params: 4,049,571 (15.45 MB)

PREGUNTA 1

El código anterior construye un clasificador de imágenes empleando transfer learning. Para lo cual emplea un modelo ya entrenado (EfficientNetB0) con el dataset de Imagenet y se añade una última capa propia para el problema específico al cual queremos adaptarlo.

- EfficientNetB0 es una red convolucional ya entrenada por Google sobre ImageNet (1.4M imágenes, 1000 categorías: perros, autos, objetos, etc.). Ya "sabe" extraer características visuales (bordes, texturas, formas).
- Bloquea los pesos del modelo base para que NO se actualicen durante el entrenamiento.

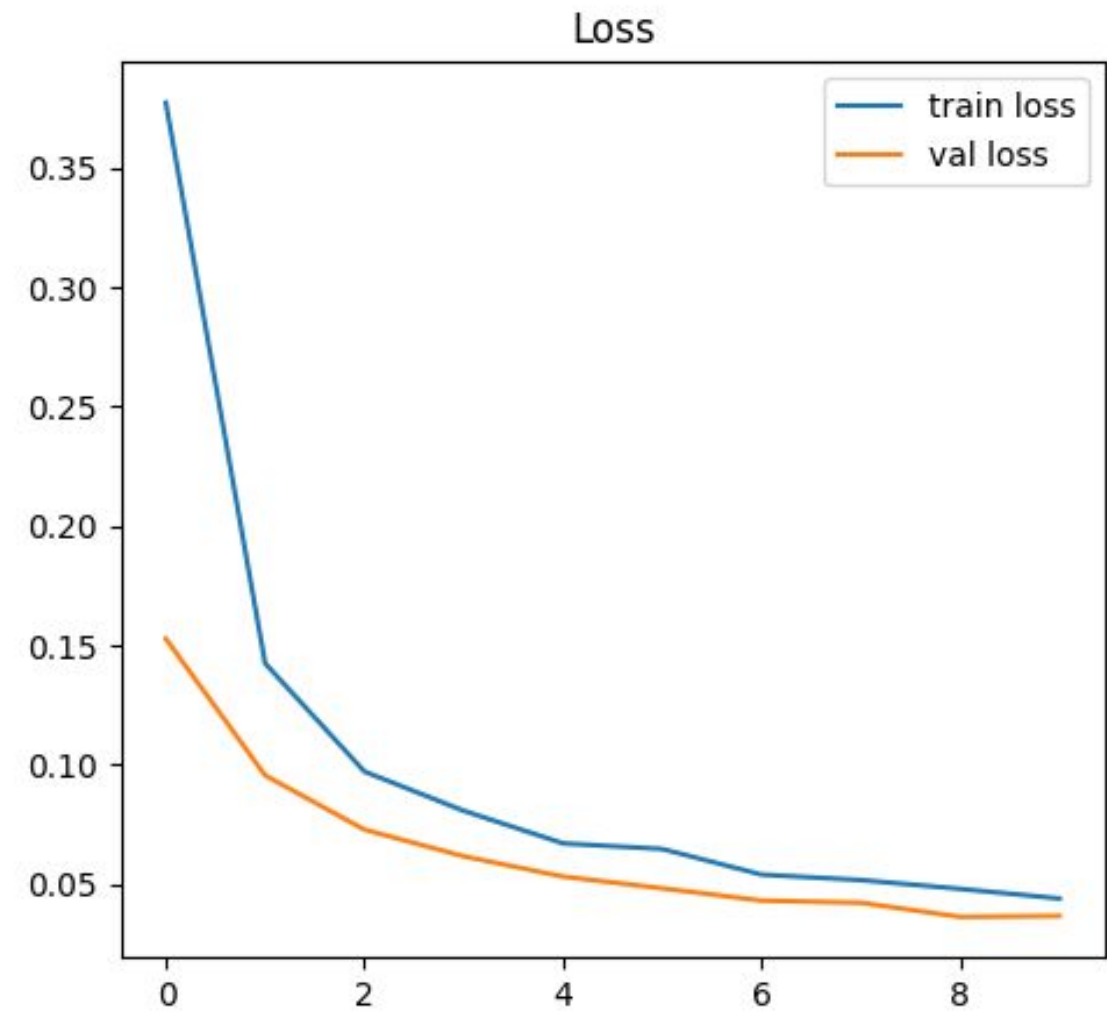
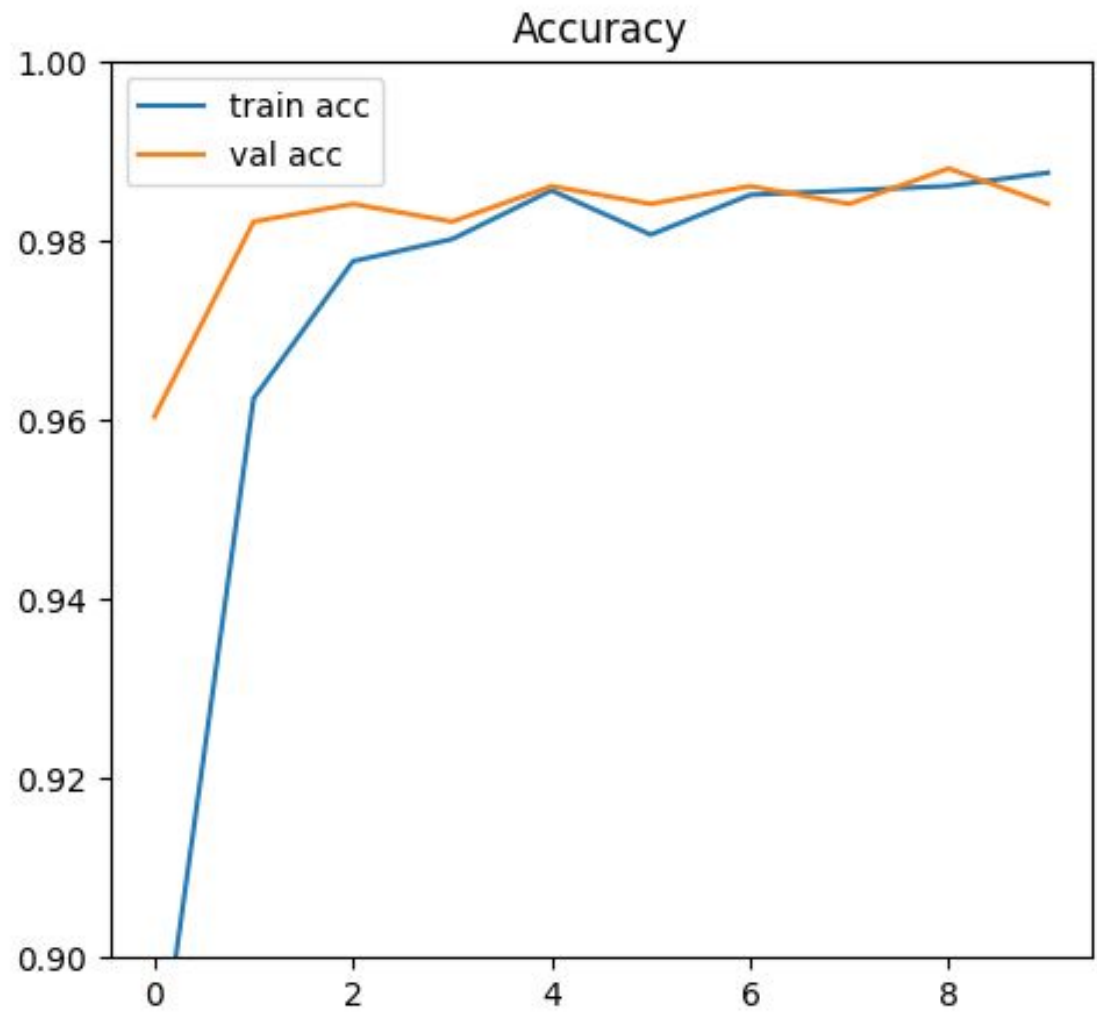
PREGUNTA 1

- `efficientnetb0 (Functional) → (None, 4, 4, 1280)`: la salida del extractor de features. Con input 128x128, EfficientNetB0 la reduce a un mapa espacial de 4x4 con 1280 canales/filtros. None es el tamaño de batch (variable, se define en tiempo de entrenamiento).
- `global_average_pooling2d → (None, 1280)`: promedia cada uno de los 1280 mapas de 4x4 a un solo número, aplanando de 3D a 1D. Sin parámetros (0) porque solo hace un promedio, no aprende nada.
- `dropout → (None, 1280)`: no cambia la forma, solo apaga neuronas al azar durante entrenamiento. 0 parámetros porque no tiene pesos, solo una regla de comportamiento.
- `dense (Dense) → (None, 3)`: capa final, 3 salidas → confirma que `class_names` tiene 3 categorías. 3,843 parámetros = 1280×3 (pesos) + 3 (bias)

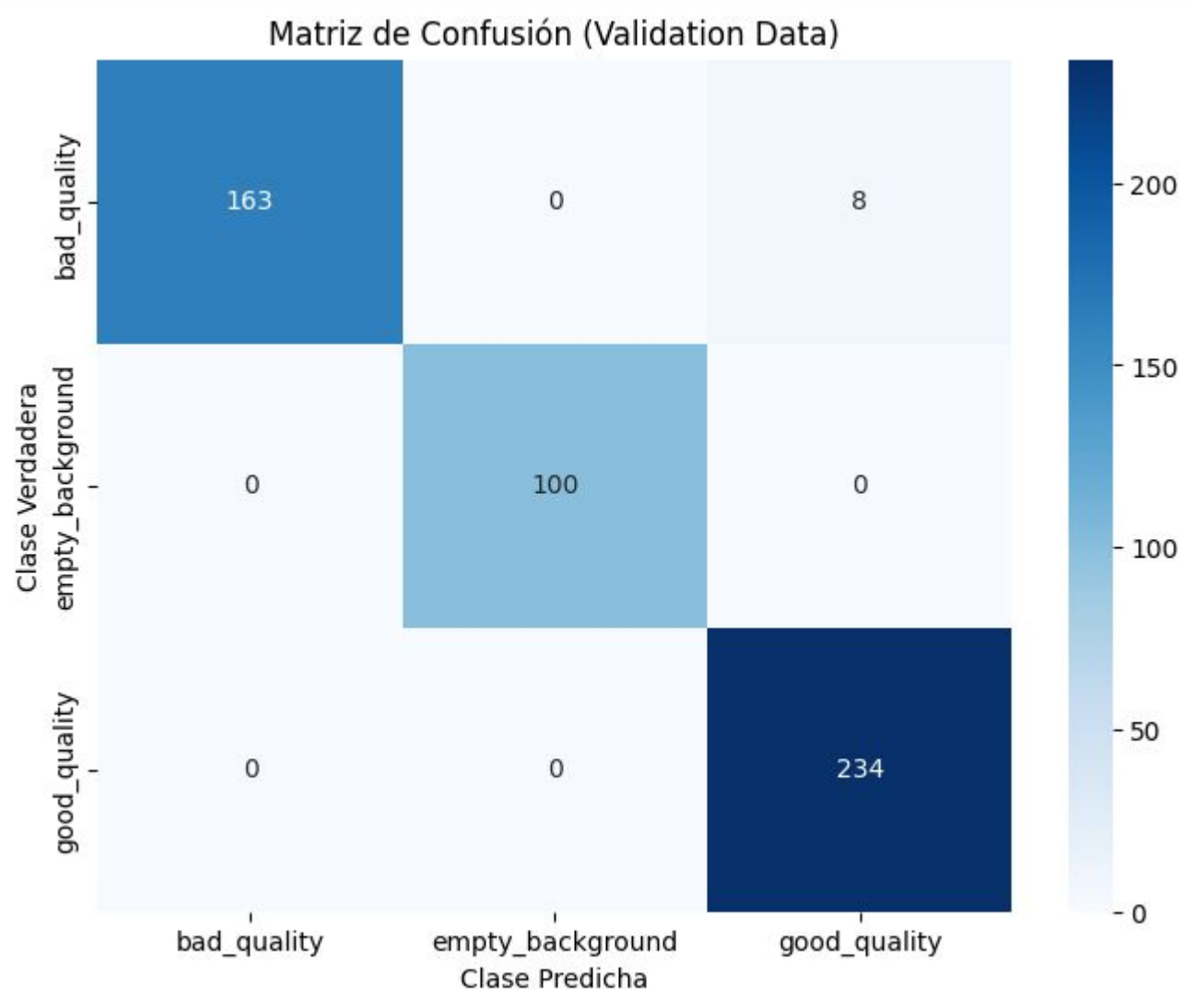
PREGUNTA 2

- Cuando ejecutamos de manera secuencial estos entrenamientos, podemos ver que el modelo sigue entrenándose desde donde se había quedado, no se reinician los pesos del modelo.
- Implicación práctica: correr el mismo bloque varias veces equivale a entrenar más épocas en total (5 + 5 + 5...), no a repetir el mismo entrenamiento desde cero. Es una forma común (aunque manual) de entrenar de forma incremental y evaluar si conviene seguir o si ya empezó a hacer overfitting.
- Lo que se puede observar adicionalmente es que a más épocas el accuracy por lo general va incrementando en el conjunto de train y también en el de validación hasta cierto punto. No siempre más épocas es mejora ya que puede quedarse como overfitting.

PREGUNTA 2



PREGUNTA 3



PREGUNTA 4

16/16 18s 597ms/step

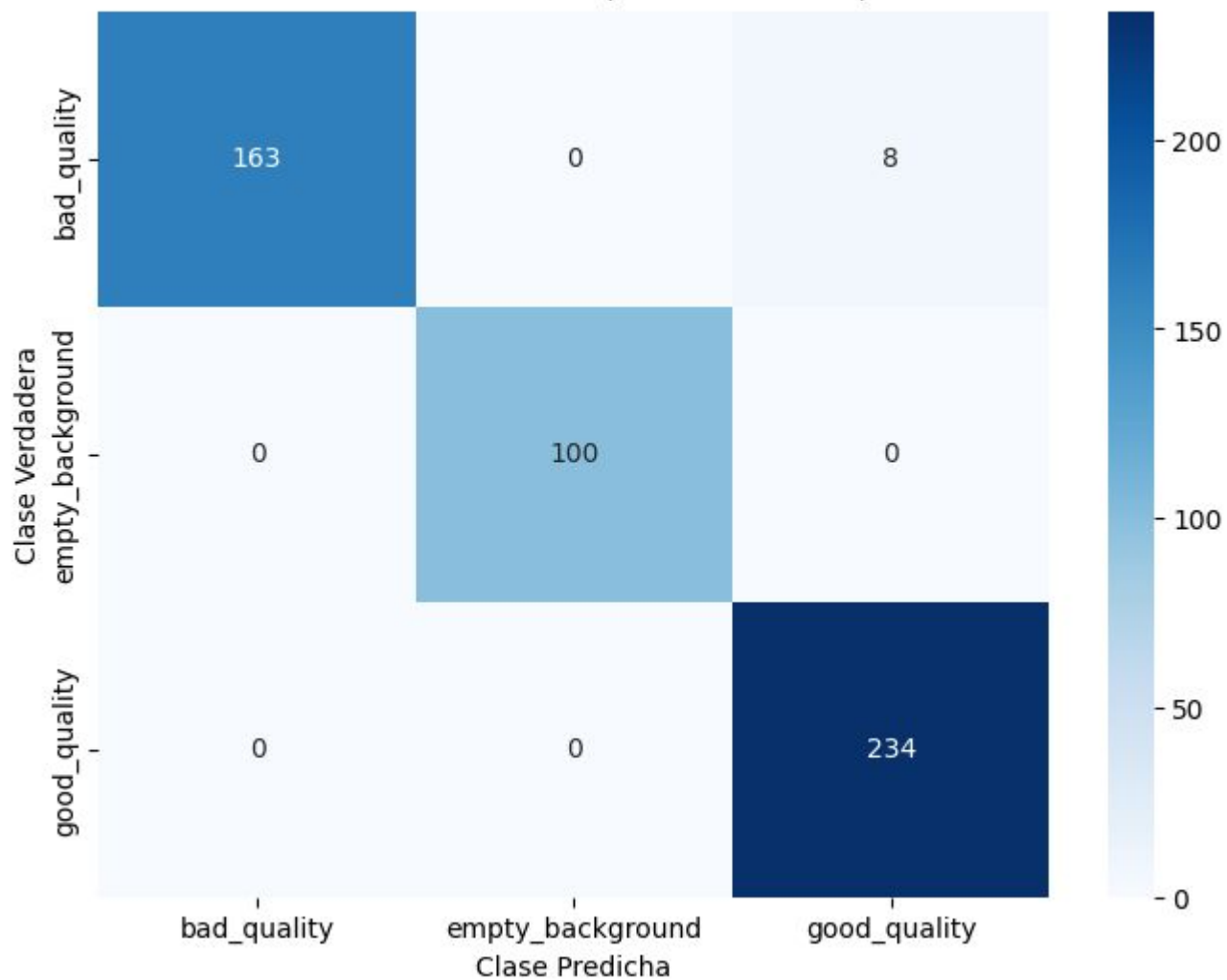
Matriz de Confusión (Validation Data):

```
[[163  0  8]
 [  0 100  0]
 [  0  0 234]]
```

Reporte de Clasificación (Validation Data):

	precision	recall	f1-score	support
bad_quality	1.00	0.95	0.98	171
empty_background	1.00	1.00	1.00	100
good_quality	0.97	1.00	0.98	234
accuracy			0.98	505
macro avg	0.99	0.98	0.99	505
weighted avg	0.98	0.98	0.98	505

Matriz de Confusión (Validation Data)



PREGUNTA 4

Matriz de Confusión (Training Data):

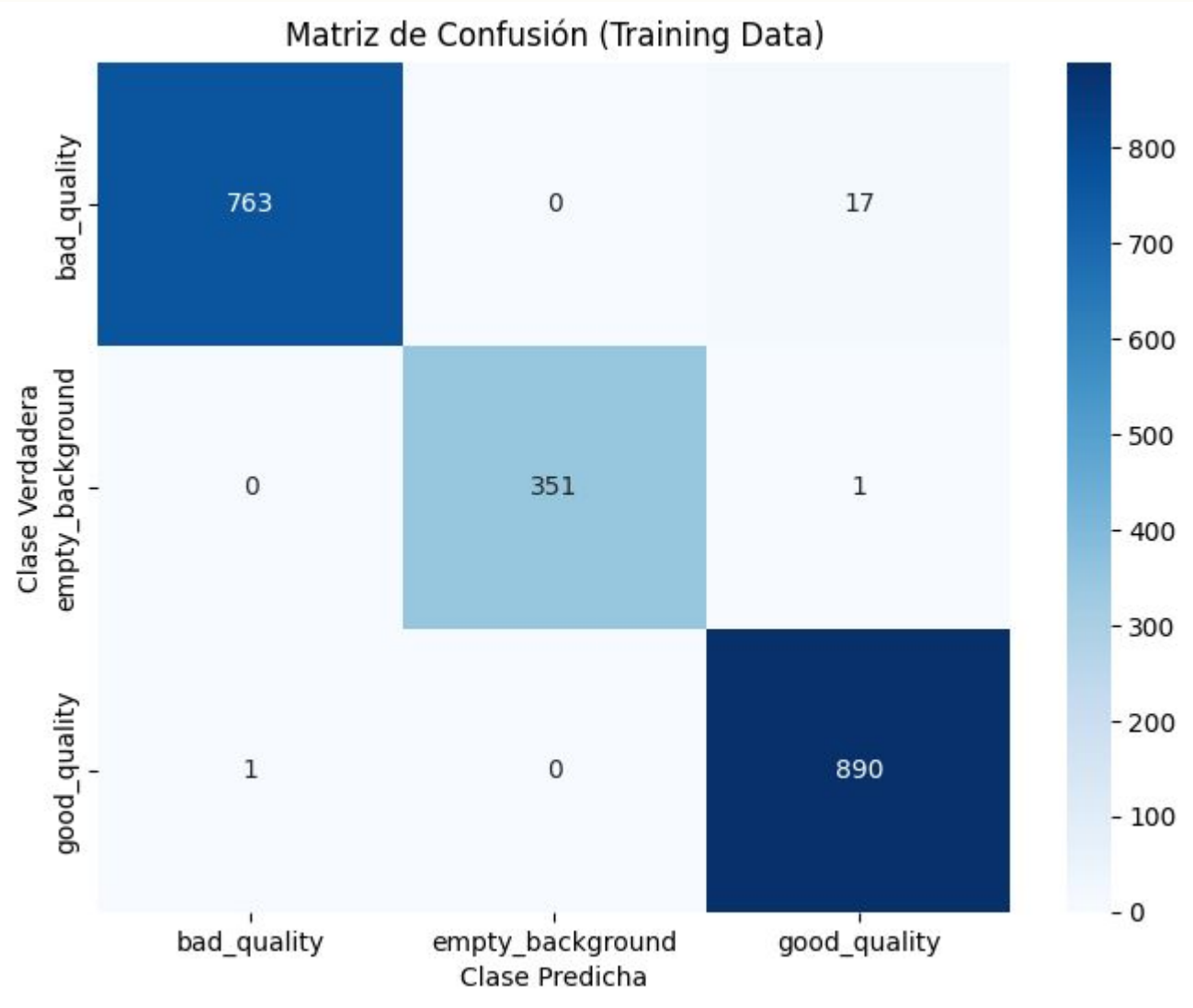
[[763 0 17]

[0 351 1]

[1 0 890]]

Reporte de Clasificación (Training Data):

	precision	recall	f1-score	support
bad_quality	1.00	0.98	0.99	780
empty_background	1.00	1.00	1.00	352
good_quality	0.98	1.00	0.99	891
accuracy			0.99	2023
macro avg	0.99	0.99	0.99	2023
weighted avg	0.99	0.99	0.99	2023



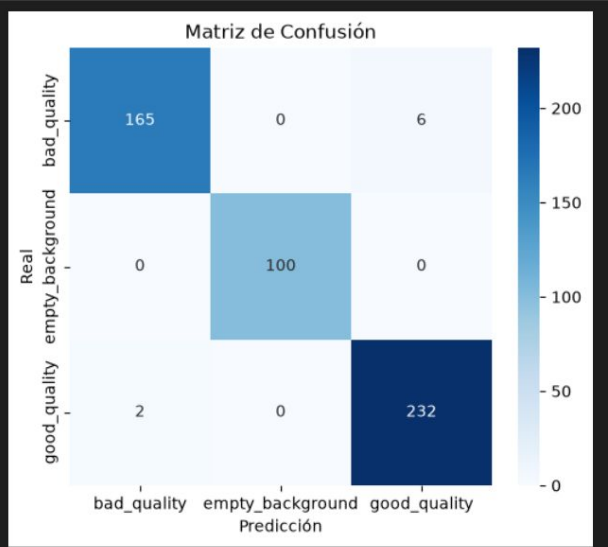
PREGUNTA 4

- No se observa un sobreajuste significativo. El modelo obtuvo una exactitud de aproximadamente 99.06 % en entrenamiento y 98.42 % en validación.
- Para la clase `bad_quality`, la precisión es aproximadamente 99.87 %, lo que indica que casi todas las imágenes clasificadas como de mala calidad realmente pertenecen a esa clase. El recall es aproximadamente 97.82 %, lo que significa que el modelo identifica correctamente casi el 98 % de los limones de mala calidad, aunque 17 imágenes de mala calidad fueron clasificadas como `good_quality`.

PREGUNTA 5

MobileNetV2

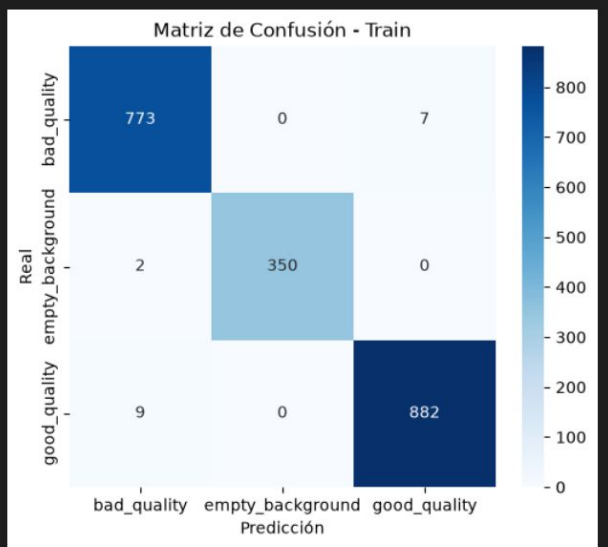
eval



	precision	recall	f1-score	support
bad_quality	0.99	0.96	0.98	171
empty_background	1.00	1.00	1.00	100
good_quality	0.97	0.99	0.98	234
accuracy			0.98	505
macro avg	0.99	0.99	0.99	505
weighted avg	0.98	0.98	0.98	505

Accuracy real calculado correctamente: 0.9842

train

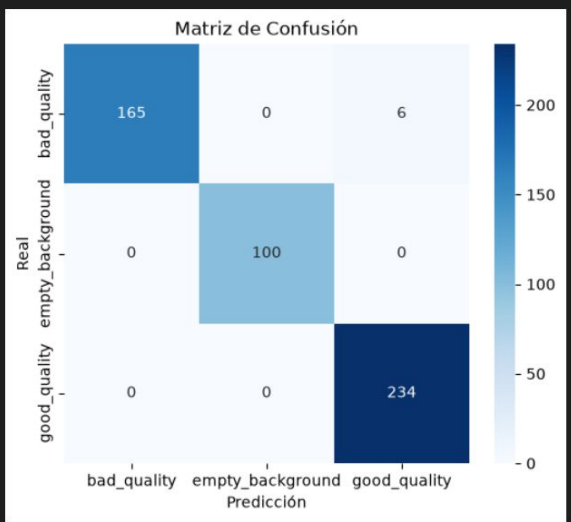


	precision	recall	f1-score	support
bad_quality	0.99	0.99	0.99	780
empty_background	1.00	0.99	1.00	352
good_quality	0.99	0.99	0.99	891
accuracy			0.99	2023
macro avg	0.99	0.99	0.99	2023
weighted avg	0.99	0.99	0.99	2023

Accuracy real en train: 0.9911

EfficientNetB0

eval

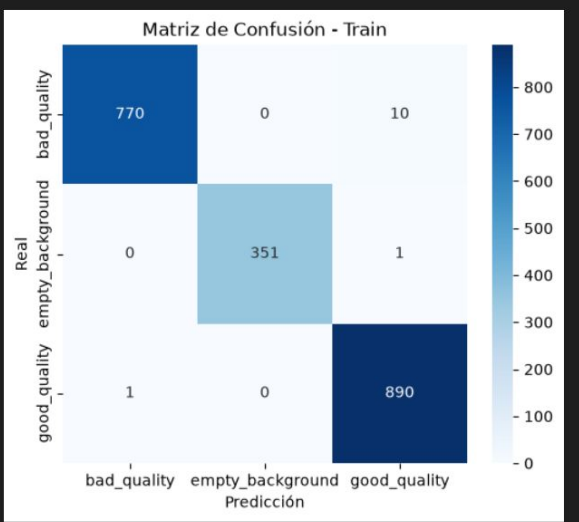


Reporte de clasificación:

	precision	recall	f1-score	support
bad_quality	1.0000	0.9649	0.9821	171
empty_background	1.0000	1.0000	1.0000	100
good_quality	0.9750	1.0000	0.9873	234
accuracy			0.9881	505
macro avg	0.9917	0.9883	0.9898	505
weighted avg	0.9884	0.9881	0.9881	505

Accuracy real calculado correctamente: 0.9881

train



Reporte de clasificación (train):

	precision	recall	f1-score	support
bad_quality	0.9987	0.9872	0.9929	780
empty_background	1.0000	0.9972	0.9986	352
good_quality	0.9878	0.9989	0.9933	891
accuracy			0.9941	2023
macro avg	0.9955	0.9944	0.9949	2023
weighted avg	0.9941	0.9941	0.9941	2023

Accuracy real calculado correctamente (train): 0.9941

PREGUNTA 5

- Ambos aciertan parecido, así que la decisión no es por precisión sino por "costo" de usarlo.
- MobileNetV2 es más liviano y rápido
- EfficientNetB0 es más "pesado"
- Si el modelo se va a usar en tiempo real o en un dispositivo con recursos limitados (celular, cámara, app), conviene MobileNetV2: mismo resultado, menos recursos.
- Si el modelo corre en un servidor sin apuros de velocidad (por ejemplo, procesamiento nocturno), la diferencia de peso no importa tanto y EfficientNetB0 deja más margen para mejorar a futuro.